
Started on Wednesday, 13 May 2026, 3:38 PM

State Finished

Completed on Wednesday, 13 May 2026, 4:35 PM

Time taken 56 mins 53 secs

Grade 6.00 out of 10.00 (60%)

Question 1 | Correct Mark 1.00 out of 1.00

Given a number, convert it into corresponding alphabet.

Input	Output
1	A
26	Z
27	AA
676	YZ

Input Format

Input is an integer

Output Format

Print the alphabets

Constraints

$1 \leq \text{num} \leq 4294967295$

Sample Input 1

26

Sample Output 1

Z

For example:

Test	Result
<code>print(excelNumber(26))</code>	Z

Answer: (penalty regime: 0 %)

Reset answer

```
1 def excelNumber(num):
2     result=""
3     while num>0:
4         num-=1
5         result=chr(65+(num%26))+result
6         num//=26
7     return result
```

	Test	Expected	Got	
✓	print(excelNumber(26))	Z	Z	✓
✓	print(excelNumber(27))	AA	AA	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 2 | Correct Mark 1.00 out of 1.00

Give a dictionary with value lists, sort the keys by summation of values in value list.

Input : test_dict = {'Gfg': [6, 7, 4], 'best': [7, 6, 5]}

Output : {'Gfg': 17, 'best': 18}

Explanation : Sorted by sum, and replaced.

Input : test_dict = {'Gfg': [8,8], 'best': [5,5]}

Output : {'best': 10, 'Gfg': 16}

Explanation : Sorted by sum, and replaced.

Sample Input:

2

Gfg 6 7 4

Best 7 6 5

Sample Output

Gfg 17

Best 18

For example:

Input	Result
2	Gfg 17
Gfg 6 7 4	Best 18
Best 7 6 5	

Answer: (penalty regime: 0 %)

```

1 n = int(input())
2 test_dict={}
3 for _ in range(n):
4     parts=input().split()
5     key=parts[0]
6     values=list(map(int,parts[1:]))
7     test_dict[key]=values
8 sorted_dict=dict(sorted(test_dict.items(),key=lambda x: sum(x[1])))
9 for key,val in sorted_dict.items():
10     print(f"{key} {sum(val)}")

```

	Input	Expected	Got	
✓	2 Gfg 6 7 4 Best 7 6 5	Gfg 17 Best 18	Gfg 17 Best 18	✓
✓	2 Gfg 6 6 Best 5 5	Best 10 Gfg 12	Best 10 Gfg 12	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 3 | Incorrect Mark 0.00 out of 1.00

Create a student dictionary for n students with the student name as key and their test mark assignment mark and lab mark as values. Do the following computations and display the result.

1. Identify the student with the highest average score
2. Identify the student who has the highest Assignment marks
3. Identify the student with the Lowest lab marks
4. Identify the student with the lowest average score

Note:

If more than one student has the same score display all the student names

Sample input:

4

James 67 89 56

Lalith 89 45 45

Ram 89 89 89

Sita 70 70 70

Sample Output:

Ram

James Ram

Lalith

Lalith

For example:

Input	Result
4	Ram
James 67 89 56	James Ram
Lalith 89 45 45	Lalith
Ram 89 89 89	Lalith
Sita 70 70 70	

Answer: (penalty regime: 0 %)

```

1 n=int(input())
2 students={}
3 for _ in range(n):
4     data=input().split()
5     name=data[0]
6     marks=list(map(int,data[1:]))
7     students[name]=marks
8 avg={n:sum(m)/3 for n,m in students.items()}
9 high_avg=max(avg.values())
10 low_avg=min(avg.values())

```

	Input	Expected	
✖	4 James 67 89 56 Lalith 89 45 45 Ram 89 89 89 Sita 70 70 70	Ram James Ram Lalith Lalith	✖

Some hidden test cases failed, too.
Your code must pass all tests to earn any marks. Try again.

Incorrect

Marks for this submission: 0.00/1.00.

Question 4 | Correct Mark 1.00 out of 1.00

In the game of Scrabble™, each letter has points associated with it. The total score of a word is the sum of the scores of its letters. More common letters are worth fewer points while less common letters are worth more points. The points associated with each letter are shown below:

Points Letters

1 A, E, I, L, N, O, R, S, T and U

2 D and G

3 B, C, M and P

4 F, H, V, W and Y

5 K

8 J and X

10 Q and Z

Write a program that computes and displays the Scrabble™ score for a word. Create a dictionary that maps from letters to point values. Then use the dictionary to compute the score.

A Scrabble™ board includes some squares that multiply the value of a letter or the value of an entire word. We will ignore these squares in this exercise.

[Sample Input](#)

REC

[Sample Output](#)

REC is worth 5 points.

For example:

Input	Result
REC	REC is worth 5 points.

Answer: (penalty regime: 0 %)

```

1 scrabble_scored= {
2     1:"A E I L N O R S T U",
3     2:"D G",
4     3:"B C M P",
5     4:"F H V W Y",
6     5:"K",
7     8:"J X",
8     10:"Q Z"
9 }
10 letter_points={}
11 for points,letters in scrabble_scored.items():
12     for letter in letters.split():
13         letter_points [letter]=points
14 word=input().upper()
15 score=0
16 for char in word:
17     score+=letter_points.get(char,0)
18 print(f"{word} is worth {score} points.")

```

	Input	Expected	Got	
✓	GOD	GOD is worth 5 points.	GOD is worth 5 points.	✓
✓	REC	REC is worth 5 points.	REC is worth 5 points.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.



Question 5 | Incorrect Mark 0.00 out of 1.00

Given an array of names of candidates in an election. A candidate name in the array represents a vote cast to the candidate. Print the name of candidates received Max vote. If there is tie, print a lexicographically smaller name.

Examples:

```
Input : votes[] = {"john", "johnny", "jackie",  
                  "johnny", "john", "jackie",  
                  "jamie", "jamie", "john",  
                  "johnny", "jamie", "johnny",  
                  "john"};
```

Output : John

We have four Candidates with name as 'John', 'Johnny', 'jamie', 'jackie'. The candidates John and Johnny get maximum votes. Since John is alphabetically smaller, we print it. Use dictionary to solve the above problem

Sample Input:

```
10  
John  
John  
Johnny  
Jamie  
Jamie  
Johnny  
Jack  
Johnny  
Johnny  
Jackie
```

Sample Output:

```
Johnny
```


Answer: (penalty regime: 0 %)

```

1 n = int(input())
2 votes = {}
3 for i in range(n):
4     name = input().split()
5     votes[name] = votes.get(name,0) + 1
6 max_count = max(votes.values())
7 result = min(name for name, count in votes.items() if count == max_count)
8 print(result)

```

	Input	Expected	Got	
✖	10 John John Johny Jamie Jamie Johny Jack Johny Johny Jackie	Johny	***Run error*** Traceback (most recent call last): File "__tester__.python3", line 5, in <module> votes[name] = votes.get(name,0) + 1 TypeError: unhashable type: 'list'	✖

Testing was aborted due to error.

Your code must pass all tests to earn any marks. Try again.

Show differences

Incorrect

Marks for this submission: 0.00/1.00.

Question 6 | Not answered Mark 0.00 out of 1.00

A company wants to send its quotation secretly to its client. The company decided to encrypt the amount they are sending to their client with some special symbols so that the equation amount will not be revealed to any external person. They used the special symbols !, @, #, \$, %, ^, &, *, >, < for 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 respectively. Write a python code to help the company to convert the amount to special symbols.

(Value rounded off to 2 decimal points)

Input

n: Float data type which reads amount to send

Output

s: : String data type which displays symbols

Sample Testcase 1

Input

10000

Output

@!!!!!!

Sample Testcase2

1234.56

Output

@#\$%^.^&

For example:

Input	Result
1345.23	@\$%^.^#\$
15000.59	@^!!!!.^<
156789	@^&*>.<.!!

Answer: (penalty regime: 0 %)

1 ||

Question 7 | Incorrect Mark 0.00 out of 1.00

Objective:

Develop a Python program that takes an input string from the user and counts the number of occurrences of each vowel (a, e, i, o, u) in the string. The program should be case-insensitive, meaning it should treat uppercase and lowercase vowels as the same.

Description:

Vowels play a significant role in the English language and other alphabet-based languages. Counting vowels in a given string is a fundamental task that can be applied in various text processing applications, including speech recognition, linguistic research, and text analysis. The objective of this problem is to create a Python script that accurately counts and displays the number of times each vowel appears in a user-provided string.

Program Requirements:**Input:**

First line reading String as input, The string can contain any characters, including letters, numbers, and special characters.

Output:

Display the number of occurrences of each vowel in the string.

The output should list each vowel followed by its count.

Example:

Consider the following example for better understanding:

- **Input:** "Python Programming"
- **Output**

```
a = 1
e = 0
i = 1
o = 2
u = 0
```

For example:

Input	Result
Hello World	a = 0 e = 1 i = 0 o = 2 u = 0
Python	a = 0 e = 0 i = 0 o = 1 u = 0

Answer: (penalty regime: 0 %)

```
1 text = input("")
2 text = text.lower()
3 vowels = {'a':0,'e':0,'i':0,'o':0,'u':0}
4 for char in vowels:
5     vowels[char]+=1
6 for vowel,count in vowels.items():
7     print(f"{vowel} = {count}")
```

	Input	Expected	Got	
✖	Hello World	a = 0 e = 1 i = 0 o = 2 u = 0	a = 1 e = 1 i = 1 o = 1 u = 1	✖
✖	AEIOU aeio u	a = 2 e = 2 i = 2 o = 2 u = 2	a = 1 e = 1 i = 1 o = 1 u = 1	✖

Some hidden test cases failed, too.
Your code must pass all tests to earn any marks. Try again.

Show differences

Incorrect

Marks for this submission: 0.00/1.00.

Question 8 | Correct Mark 1.00 out of 1.00

A sentence is a string of single-space separated words where each word consists only of lowercase letters. A word is uncommon if it appears exactly once in one of the sentences, and does not appear in the other sentence.

Given two sentences s_1 and s_2 , return a list of all the uncommon words. You may return the answer in any order.

Example 1:

Input: s_1 = "this apple is sweet", s_2 = "this apple is sour"

Output: ["sweet", "sour"]

Example 2:

Input: s_1 = "apple apple", s_2 = "banana"

Output: ["banana"]

Constraints:

$1 \leq s_1.length, s_2.length \leq 200$

s_1 and s_2 consist of lowercase English letters and spaces.

s_1 and s_2 do not have leading or trailing spaces.

All the words in s_1 and s_2 are separated by a single space.

Note:

Use dictionary to solve the problem

For example:

Input	Result
this apple is sweet this apple is sour	sweet sour

Answer: (penalty regime: 0 %)

```

1 def uncommonFromSentences(s1,s2):
2     words=s1.split()+s2.split()
3     count={}
4     for word in words:
5         count[word]=count.get(word,0)+1
6     return[word for word in count if count[word]==1]
7 s1=input()
8 s2=input()
9 print(*uncommonFromSentences(s1,s2))

```

	Input	Expected	Got	
✓	this apple is sweet this apple is sour	sweet sour	sweet sour	✓
✓	apple apple banana	banana	banana	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 9 | Correct Mark 1.00 out of 1.00

You are given a string **word**. A letter is called **special** if it appears both in lowercase and uppercase in **word**.

Your task is to return the number of **special** letters in **word**.

Constraints

- The input string **word** will contain only alphabetic characters (both lowercase and uppercase).
- The solution must utilize a dictionary to determine the number of special letters.
- The function should handle various edge cases, such as strings without any special letters, strings with only lowercase or uppercase letters, and mixed strings.

Examples

Example 1:

Input: word = "aaAbcBC"

Output: 3

Explanation:

The special characters in `word` are 'a', 'b', and 'c'.

Example 2:

Input: word = "abc"

Output: 0

Explanation:

No character in `word` appears in uppercase.

For example:

Test	Result
<code>print(count_special_letters("AaBbCcDdEe"))</code>	5

Answer: (penalty regime: 0 %)

[Reset answer](#)

```
1 def count_special_letters(word: str) -> int:
2     lower = set()
3     upper = set()
4     for ch in word:
5         if ch.islower():
6             lower.add(ch)
7         elif ch.isupper():
8             upper.add(ch.lower())
9     special = lower.intersection(upper)
10    return len(special)
```

	Test	Expected	Got	
✓	<code>print(count_special_letters("AaBbCcDdEe"))</code>	5	5	✓
✓	<code>print(count_special_letters("ABCDE"))</code>	0	0	✓
✓	<code>print(count_special_letters("abcde"))</code>	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.



Question 10 | Correct Mark 1.00 out of 1.00

A sentence is a list of words that are separated by a single space with no leading or trailing spaces. Each word consists of lowercase and uppercase English letters.

A sentence can be shuffled by appending the 1-indexed word position to each word then rearranging the words in the sentence.

For example, the sentence "This is a sentence" can be shuffled as "sentence4 a3 is2 This1" or "is2 sentence4 This1 a3".

Given a shuffled sentence *s* containing no more than 9 words, reconstruct and return the original sentence.

Example 1:

Input:

is2 sentence4 This1 a3

Output:

This is a sentence

Explanation: Sort the words in *s* to their original positions "This1 is2 a3 sentence4", then remove the numbers.

Example 2:

Input:

Myself2 Me1 I4 and3

Output:

Me Myself and I

Explanation: Sort the words in *s* to their original positions "Me1 Myself2 and3 I4", then remove the numbers.

Constraints:

$2 \leq s.length \leq 200$

s consists of lowercase and uppercase English letters, spaces, and digits from 1 to 9.

The number of words in *s* is between 1 and 9.

The words in *s* are separated by a single space.

s contains no leading or trailing spaces.

Answer: (penalty regime: 0 %)

```
1 def sortSentence(s):
2     words=s.split()
3     sorted_words=['']*len(words)
4     for word in words:
5         position=int(word[-1])-1
6         sorted_words[position]=word[:-1]
7     return ' '.join(sorted_words)
8 s = input()
9 print(sortSentence(s))
```

	Input	Expected	Got	
✓	is2 sentence4 This1 a3	This is a sentence	This is a sentence	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.